Bootstrapping Vision-Language-Action Models for New Robot Embodiments with Staged Reinforcement Learning

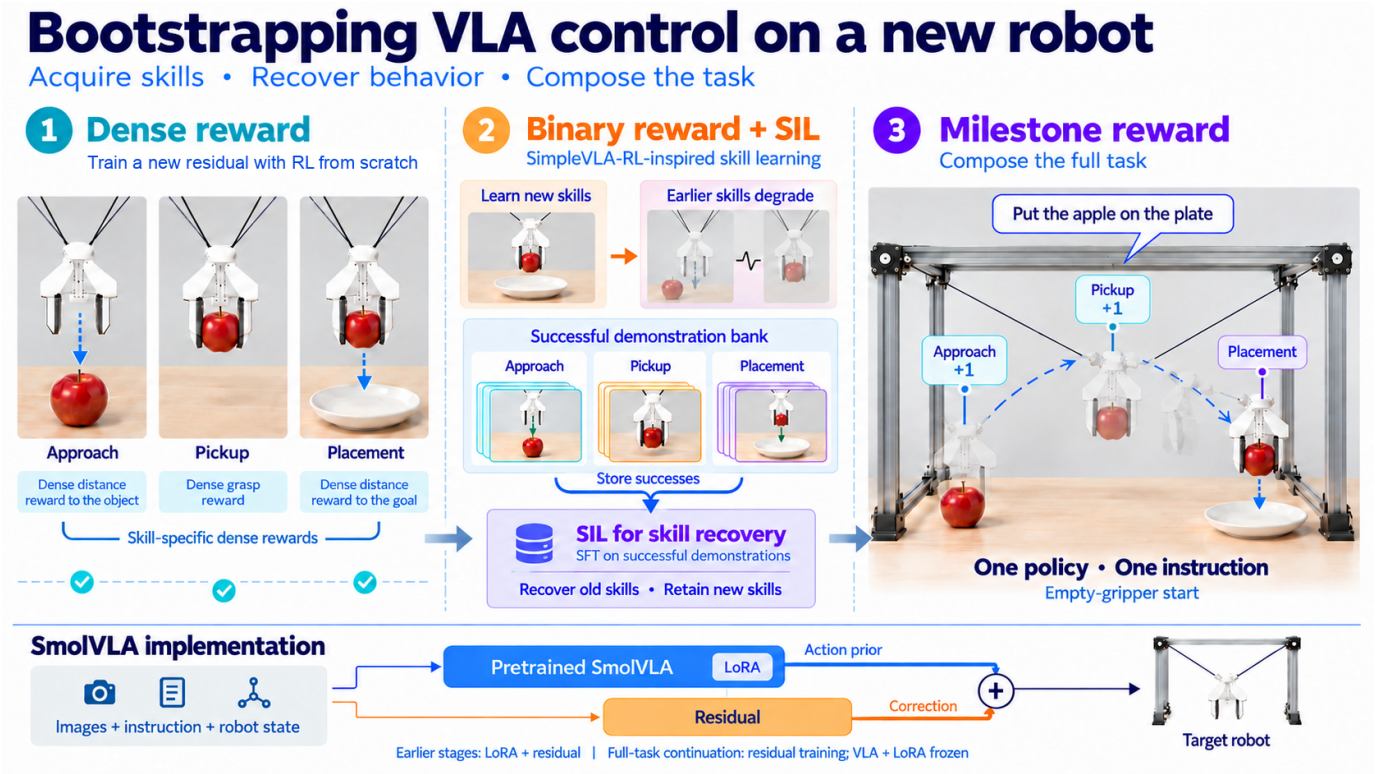


Fig. 1. Training pipeline. Dense rewards guide individual skills. Binary refinement and self-imitation consolidate them. Milestone rewards then train one policy to execute the complete instruction from an empty-gripper start.

***Abstract*—Can a pretrained vision-language-action model learn to pick and place on a new robot without human demonstrations on that robot? We retain the pretrained action expert and train a residual controller in simulation. We divide the instruction into skills for which progress is easy to measure. Dense rewards teach the first movements, binary rewards refine each skill, and self-imitation on saved successes helps recover skills lost during training. We then relabel trajectories with one full-task instruction and reuse the skill success conditions as milestones for reinforcement learning. On a four-cable robot, training also used scripted and assisted trajectories. The policy completed 1,416 of 2,048 unassisted development attempts (69.14%). On a simulated WidowX-200, placement from an already held object succeeded in 35.64% of attempts, but completion from an empty gripper reached only 7.13%. The difference illustrates why composition must be tested from the start of the task.**

***Index Terms*—Robot learning, reinforcement learning, manipulation, vision-language-action models, robot adaptation.**

I. Introduction

VLA models provide an ability to perform complex object-manipulation tasks based on images received from cameras on the robot. VLA models are often trained on large manually collected datasets. For example, OpenVLA was pretrained on 970,000 robot demonstrations [1]. Modern approaches use RL to enhance the success rate up to 100% for pretrained VLA models on selected tasks [2]. The base VLA model can be adapted through parameter-efficient fine-tuning. Some VLAs can control even two manipulators in parallel [3]. But what if we need to adapt a VLA model to a dissimilar robot embodiment with different degrees of freedom, such as a cable-driven robot with a gripper suspended by four cables, three translation axes, rotation about the vertical axis, and gripper opening? We need to adapt the part of the VLA which produces actions to the target control space. Changing the number of output channels gives commands the robot can execute, but does not by itself teach the robot to grasp, carry, and release an object. Supervised training requires a dataset even when training only a part of the whole model. But we want to generalize this process instead of creating a new dataset with the same tasks for new robots. Can we use RL algorithms to train a VLA's new action module from scratch? In this work, we adapt the pretrained action expert through LoRA and train a newly initialized MLP residual controller over its outputs.

A newly initialized action module does not initially provide

reliable task-directed behavior, which is why using terminal-only rewards will barely give any signal for RL to change weights when successful attempts are rare. At the same time, dense rewards for complex object-manipulation tasks are difficult to design. Before grasping, the robot needs to move toward the object; after grasping, it needs to lift the object away from its initial position. We divide a complex instruction into sequential skills whose progress can be described by a dense reward. RL can produce a useful signal for the model at this point, and we can achieve an acceptable success rate and then increase it with a binary reward [4]. Uniting n skills, each having a conditional success rate p_i given completion of the preceding skills under the same instruction, gives a total success rate $p_{\text{total}} = \prod_{i=1}^{n} p_i$. But each skill used its own text instruction, so we may record the most successful demonstrations of each skill, relabel them under one instruction, and concatenate segments whose ending and starting states are compatible. This method of self-imitation learning allows us to train the composed task using successful skill behavior. Moreover, it aligns with the SimpleVLA-RL pipeline, and we can then enhance the model with a binary reward. The success rates of isolated skills do not directly determine the success rate under the composed instruction, because both the instruction and the states at skill transitions change.

So far, we have success criteria for each skill, and we can increase the availability of a learning signal for GRPO's groups by separating the reward into phases and rewarding completion of each phase. An attempt which lifts the object and then drops it can provide a useful comparison with an attempt which never grasps it, although both fail the complete instruction. In general, such a pipeline may allow us to adapt VLA models to unseen robots by describing each skill without manually generating demonstration datasets. Our experiments use simulation, and the CDPR training also includes scripted and assisted trajectories. We also provide a second robot manipulator, WidowX-200, and clarify our pipeline on it. The reported results therefore concern adaptation without collecting human demonstrations on the target robot, while still using automatically generated supervision.

II. Related Work

GRPO provides an effective way to tune a model's weights. It compares rewards within a group of attempts at the same prompt. But it requires informative groups, which are unlikely to be obtained from a controller with little initial task ability. If all attempts receive the same reward, their group-relative advantages are zero [5]. Dynamic sampling filters groups and collects additional samples to maintain an informative training batch [6]. Filtering cannot create successful behavior when all sampled attempts fail. In our setting, we also change the starting states and use dense feedback during initial skill acquisition.

SimpleVLA-RL applies group-relative updates and dynamic sampling to VLA rollouts. Its experiments also show why the initial policy matters: on five RoboTwin 2.0 tasks, RL increased average success from 7.3% to 25.4% after supervised fine-tuning on 100 demonstrations per task, while success remained zero without task-specific fine-tuning [4]. We use separately practiced skills to obtain useful behavior before learning the complete instruction.

iRe-VLA alternates RL on an action head with supervised learning on expert demonstrations and successful online trajectories. During RL, it freezes the vision-language model; supervised learning then updates the model using both earlier and newly collected data [7]. We also alternate interaction and supervised learning, but our skill-recovery bank contains successes from earlier and recent policies on the target robot. The supervised update helps recover skills which degrade while the shared controller learns another skill.

VLA models rely on vision and proprioception, but successful behavior on familiar robots does not establish that the model can use these observations effectively on a new embodiment. PLD provides lightweight residual specialists over a frozen VLA, collects trajectories through policy-aligned residual interventions, and distills the resulting data into the generalist [2]. This allows the generalist to learn from behavior generated by residual RL. We retain a residual controller on the target robot and use successful skill trajectories both for recovery and for learning one composed instruction.

We use self-imitation learning in the sense of learning from the policy's own past successful behavior. Our supervised action-regression loss differs from the actor-critic objective of the original Self-Imitation Learning method [8].

III. Methods

The goal is to adapt a pretrained VLA to complete language-conditioned manipulation tasks on a target robot without collecting human demonstrations. We study multi-stage pick-and-place tasks: an instruction such as *put the apple on the plate* requires approach, grasp, lift, transport, and release. The policy receives camera images, proprioception, and an instruction, predicts an action chunk, and executes a prefix before observing again.

During acquisition, each skill has its own instruction, success condition, and starting states. During composition and final evaluation, the robot starts with an empty gripper and receives one complete-task instruction. This distinction prevents isolated skill performance from being treated as full-task success. Algorithm 1 summarizes the pipeline: acquire simple skills, consolidate them, and learn the complete instruction.

We use SmolVLA as the pretrained model [9]. On the CDPR and WidowX, we map its action prior to three translation commands, yaw, and gripper opening. A newly initialized residual MLP receives the mapped prior and observation features and predicts corrections to an eight-action chunk. The robot executes four actions before observing again. Earlier training also adapts the pretrained expert with low-rank adapters. The final stage freezes the expert and its adapters and continues training the retained residual controller.

A. Obtaining the first successful attempts

We start from simple skills which can be described with a dense reward that measures progress before successful

Algorithm 1 Staged VLA adaptation

1: Load the pretrained VLA, initialize the residual controller, and create successful-trajectory banks.
2: Train approach, pickup, and placement separately with dense rewards, using each skill's instruction and reset conditions.
3: After each skill, evaluate the earlier skills. If they have degraded, fit the shared policy to a balanced bank of earlier and recent successes and evaluate it again.
4: Refine the skills with binary completion rewards, repeating the retention checks and recovery when needed.
5: Relabel successful skill trajectories with the complete instruction. Collect compatible segments for composition and fit their actions under that instruction.
6: Train the complete task from empty-gripper starts with milestone feedback, and evaluate complete-task success.
7: If composition plateaus and the budget permits, freeze the VLA and its adapters, retain the learned residual, reset the optimizer, and continue full-task RL with the same milestone rewards. In the CDPR continuation, likelihoods are evaluated on stored pre-clipping samples conditional on their recorded episode offsets.

completion. For each skill, we introduce a stepwise schedule that distances the robot's starting position from the successful completion of the task, provided that a certain success rate is achieved. For bowl placement on the CDPR, the distance cap increases from 0.10 to 0.13, 0.16, and 0.19 m. Together with DAPO-style dynamic sampling, this increases the opportunity to obtain informative sampled groups, without guaranteeing that every sampled group is informative.

For *move to the apple*, progress is measured by the distance between the gripper and the object. For *pick up the apple*, proximity, grasp, and lift distinguish an approach from a completed pickup. Placement starts with an already held object, so the robot can practice transport and release before it can reliably pick up the object itself. Dense rewards allow two failed attempts to contribute to an update when one makes more progress than the other. Once successful attempts are available, binary refinement rewards completion of the skill. For a fixed scene, let p be the probability of success on one attempt and G the number of independent attempts in a group. A binary group is informative when it contains at least one success and one failure. Its probability is

$$P_{\text{mixed}} = 1 - (1-p)^G - p^G. \tag{1}$$

The two subtracted terms are the probabilities that all attempts fail or all attempts succeed. At $p = 0.03$ and $G = 8$, only 21.6% of groups contain both outcomes. Dynamic sampling filters uniform groups and collects additional groups within the rollout budget. With dense feedback, the filtering criterion is variation in return; with binary feedback, it is variation in success.

But the frequency of informative GRPO groups differs from average task success. The expected binary reward is p, whereas P_{mixed} is highest at $p = 0.5$ and falls as success approaches one. With binary rewards, group normalization also changes how much each scene contributes to the expected on-policy GRPO update. When scenes share a policy, the resulting objective can favor a small gain in one scene over a larger loss in another [10, Sec. 6].

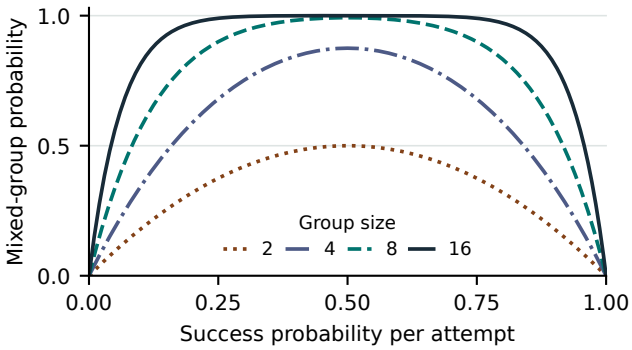


Fig. 2. Mixed-outcome probability for group sizes $G = 2, 4, 8, 16$. The curves describe independent binary attempts at a fixed scene.

For example, consider two equally likely scenes and groups of eight attempts. Suppose a policy change raises success in the first scene from 1% to 2% and lowers it in the second from 50% to 48.8%. The expected fraction of mixed groups rises from 53.47% to 57.06%, and the finite-group GRPO objective rises from 0.5381 to 0.5399. Mean task success falls from 25.5% to 25.4%.

The numbers illustrate the unfiltered on-policy binary-reward model in [10, Sec. 6] in the zero-stabilizer limit.

We also ran additional validations on LIBERO, comparing terminal-only, phase-only, and combined dense-and-phase rewards. These validations examine complete benchmark instructions. They do not directly measure the success of the separately trained skills in our adaptation pipeline. We will conduct more experiments which show separate skill success rates.

The combined LIBERO reward uses a manually defined sequence of reach, grasp, lift, move, and place phases. For each phase, a progress measure is normalized to the interval from zero to one. Reaching uses gripper–object distance, lifting uses height gained above the initial object position, and transport uses planar object–target distance. Grasping also uses gripper closure and grasp state; placement uses target proximity, release, and environment success. For current progress q_t and the previous maximum q_{best}, the rewarded increment is $\max(q_t - q_{\text{best}}, 0)$. A completion predicate advances the tracker and adds a phase bonus. The bounded progress increment and phase bonus are combined with terminal feedback, while the native task-success verifier remains unchanged. The phase-only condition sets the continuous-progress weight to zero.

This reward construction requires a progress function, normalization, and relevant state variables for every phase. In our adaptation pipeline, we use dense feedback for the initial simple skills and reuse their binary completion predicates during full-task training.

B. Retaining skills through self-imitation

After collecting successful trajectories for individual skills, we relabel them with a composed instruction and then start self-imitation learning—when the model learns on episodes generated by itself. We also smooth demonstrations with a

moving average to get rid of sharp actions. Some demonstrations become unsuccessful; we remove them from the SIL bank. For the CDPR skill bank, the moving average spans five decisions. We replay the smoothed actions from the recorded reset and retain only trajectories which still satisfy the skill's success condition. The bank stores observations from replay together with the actions that produced them.

We balance earlier and recent successful trajectories by instruction. The supervised objective is the squared error between predicted and recorded actions. After fitting, we evaluate every skill again, because an improvement in placement can conceal a loss of pickup when only the average success rate is reported. The bank may contain trajectories from several earlier policies; it is not restricted to the current policy's latest episodes.

C. Composing skills under one instruction

Relabeling changes the instruction associated with each segment. A reaching segment originally recorded under *move to the apple* is then used to teach the same first movement under *put the apple on the plate*. Concatenating segments additionally requires compatible states: pickup must begin where approach ended, and placement must begin with the grasp produced by pickup. In the CDPR experiments, we collected such segments with scripted assistance at skill transitions. Training also used complete demonstrations generated by a scripted simulator controller. These assisted and scripted data are distinguished from the policy's own unassisted successes.

For ordered skill-success events $E_1, \ldots, E_n$ under one instruction, the chain rule gives

$$P(\text{complete}) = \prod_{i=1}^{n} P(E_i \mid E_1, \ldots, E_{i-1}), \tag{2}$$

where the first factor is $P(E_1)$. The placement factor therefore depends on the grasp actually produced during the episode. An isolated placement rate measured from a prepared grasp under a placement instruction cannot be substituted for this conditional probability. Self-imitation teaches the actions under the composed instruction, but does not guarantee a particular composed success rate.

D. Learning from partial completion

At the last training stage, we use a sparse phased reward based on the completion condition of each skill. During CDPR full-task RL, the first milestone requires reaching within 3 cm of the grasp point or establishing a grasp. The second requires a held lift of at least 5 cm. The third requires strict placement. The robot starts with an empty gripper and receives the complete instruction.

For eight attempts at the same scene, we compare outcomes separately at each milestone. Actions receive the group-relative advantage for their current stage. A successful placement also credits the earlier actions. When placement fails, an achieved approach or pickup keeps its local credit. For example, an attempt which lifts the apple and drops it and an attempt which never grasps it both fail the terminal test, but their pickup outcomes differ. A uniform placement outcome provides no local advantage for release. The WidowX implementation uses the fraction of milestones reached as its staged return.

The same distinction between informative groups and task success applies to each phase. A mixed pickup group provides a comparison for learning pickup, but does not by itself establish an improvement in complete-task success. We therefore report both milestone completion and the strict full-task result.

IV. Experiments

We tested the pipeline on a simulated CDPR and a simulated five-degree-of-freedom WidowX-200 arm. The CDPR uses MuJoCo Warp/MJ-Lab with four objects: apple, orange, potato, and tomato. The destinations are a plate and a bowl. Skill evaluations use each skill's prompt and prescribed starting states. Complete-task evaluations use one instruction and an empty gripper. A separate LIBERO-Spatial experiment examines the reward used for fine-tuning OpenVLA-OFT.

A. Skill acquisition and recovery

We noticed that learning the *pick up* skill immediately after *move to* failed. Only after learning *put into*, which moves an already held object, did the VLA start to learn *pick up*.

When training the next skill, we noticed that previously learned skills degraded drastically. We started to collect successful episodes for each skill in a SIL bank and conducted skill recovery after learning a new skill. On the CDPR, reaching success fell from 64.1% to 5.3% after training placement. In a separate recovery evaluation, fitting earlier reaching and recent placement successes raised reaching from 8.0% to 31.1%. Plate placement rose from 50.0% to 52.3%, and bowl placement from 23.1% to 27.7%. These recovery measurements use different evaluation rounds from the forgetting comparison.

When pickup examples were added to the CDPR bank, pickup recovered from zero to 9.6%. The later shared policy reached 47.8% on reaching, 11.9% on pickup, 73.8% on plate placement, and 53.5% on bowl placement. These are isolated skill evaluations. The placement rates use held-object starts and do not measure completion from an empty gripper.

On WidowX, dense placement training reduced pickup from 140/1,024 successful attempts to zero. Later, two rounds of balanced self-imitation restored reaching to 372/1,024, but pickup succeeded only once. During binary pickup training, most successes came from assisted starts with an already lifted object. In 55 of 57 updates, there was no successful pickup from an ordinary approach. The controller could therefore receive successful pickup labels without regularly practicing the transition from approach to grasp.

B. Execution under one instruction

For the complete CDPR task, we evaluated 512 development scenes outside the training validation panel, with four attempts per scene. Each attempt allowed up to 128 decisions. Strict success required the robot to grasp the object, lift it while held, and deliberately release it at the correct destination without a

(a) Cable-driven robot

	Reach	Pickup	Plate	Bowl
Skill references	64.1	26.0	71.0	32.0
After placement RL	5.3	–	–	–
Before SIL	8.0	–	50.0	23.1
After SIL	31.1	0.0	52.3	27.7
Add pickup to SIL	32.0	9.6	57.4	27.2
Later shared policy	47.8	11.9	73.8	53.5

(b) WidowX-200

	Reach	Pickup	Plate	Bowl
Dense pickup	39.6	13.7	–	–
Dense placement	5.4	0.0	73.8	60.8
SIL trial (rejected)	13.6	0.2	55.8	34.6
Binary reach	53.8	0.0	43.1	19.8
Binary pickup	0.0	0.0	24.2	11.9
After SIL	36.3	0.1	65.3	39.2

Fig. 3. Skill success (%) during acquisition and recovery. CDPR references are separately trained skill policies. The horizontal line separates forgetting and recovery evaluations, which used different rounds. Dashes indicate unavailable measurements. On WidowX, reaching and pickup use 1,024 attempts each, plate placement uses 504, and bowl placement uses 520. Placement starts with the object held on both robots. The rejected WidowX SIL trial was evaluated but was not accepted as the next policy.

carry slip or wrong placement. Numerical failures counted as failed attempts. Evaluation used only the learned controller, while training included scripted and assisted trajectories.

TABLE I
FULL-TASK PLACEMENT IN SIMULATION UNDER ONE INSTRUCTION. THE CDPR STARTS WITH AN UNASSISTED APPROACH. WIDOWX STARTS WITH THE EMPTY GRIPPER ALIGNED ABOVE THE OBJECT.

Robot	Destination	Completions / attempts	Success (%)
CDPR	Plate	774 / 1,028	75.29
	Bowl	642 / 1,020	62.94
	Both	1,416 / 2,048	69.14
WidowX-200	Plate	41 / 504	8.13
	Bowl	32 / 520	6.15
	Both	73 / 1,024	7.13

Under the full instruction, the CDPR policy lifted the object in 1,832 of 2,048 attempts and completed 1,416. Thus 416 attempts failed after the robot had already achieved pickup. Placement was more often successful on the plate than in the bowl.

The WidowX composition bank contained relabeled segments from 452 policy-generated episodes and no continuous successful chain. The policy placed an already held object in 365/1,024 attempts (35.64%), but completed the task from an empty gripper in only 73/1,024 attempts (7.13%). The difference shows why a held-object placement rate does not establish that the robot can compose pickup and placement.

Supervised composition did not by itself establish improved execution. On the CDPR, the supervised fit under the complete instruction tied its initializer at five completions in 128 attempts. On WidowX, complete-task success was 11/1,024 before composition fitting, 8/1,024 after fitting, and 73/1,024 after joint RL. Fitting stored actions under the complete instruction therefore needs to be checked by executing that instruction from the actual starting state.

C. How does intermediate feedback affect learning?

For the LIBERO reward comparison, we fine-tuned OpenVLA-OFT with SimpleVLA-RL on ten LIBERO-Spatial tasks. Each suite run began at 61.0% success and used 300 rollouts per evaluation. We kept the native benchmark success criterion. Terminal feedback rewards complete-task success, phase feedback additionally rewards milestone completion, and dense feedback additionally rewards improvement within the current phase.

TABLE II
LIBERO-SPATIAL SUITE ENDPOINTS. WEIGHTS APPLY TO CONTINUOUS PROGRESS (w_p), PHASE COMPLETION (w_c), AND THE EXTERNAL TERMINAL VERIFIER (λ). SUCCESS IS THE NATIVE BENCHMARK METRIC.

Feedback	w_p	w_c	λ	Updates	Success (%)
Terminal only	0	0	5	160	61.7
Dense + phase	.05	.05	5	160	65.3
Phase only*	0	.20	1	110	62.3
Dense + phase	.05	.05	1	110	61.0

*Phase-only shaping retains the terminal verifier.

With the same terminal weight and 160 updates, dense plus phase feedback reached 65.3%, compared with 61.7% for terminal feedback alone. On a separate task starting at 2% success, dense feedback reached 18% after 20 updates and terminal feedback reached 11%. Both runs used the same group filter. These observations show higher measured complete-task success with dense feedback in these comparisons; they do not isolate an improvement in individual skills. The 110-update comparison cannot isolate the effect of continuous progress because the phase weight also changes.

Applying only a binary terminal reward to the CDPR also produced successful placements. The terminal-only and phased-reward runs used different starting states, supervision, and success criteria, so the current experiments do not establish a causal gain from switching to a phased reward. Such a comparison requires the same initializer, resets, termination rule, and interaction budget.

All robot results are from simulation. The CDPR repeats measure rollout variation for one trained policy lineage on development scenes used for model selection. The LIBERO comparisons report one endpoint per run. Independent training runs, the reserved final-test scenes, and physical execution remain untested. The contribution of scripted composition data also remains to be isolated. The results support the use of skill acquisition, recovery, and explicit transition evaluation, while the weak WidowX pickup result shows that separately learned placement cannot compensate for an unreliable earlier skill.

V. Conclusion

The method begins with skills whose progress the simulator can measure before a complete success occurs. Successful skill trajectories then supply action targets for rehearsal and for learning the complete instruction. Finally, the same skill predicates provide intermediate feedback when the robot acts from an empty gripper. This produced unassisted pick-and-place on the CDPR using automatically generated training data. On WidowX, the policy often failed before it could use the placement skill it had learned from held-object starts. The reported rates are from simulation, and the CDPR development trials repeat one trained policy across scenes. Transfer to hardware remains to be tested.

References

[1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024. [Online]. Available: https://arxiv.org/abs/2406.09246

[2] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu, "Self-improving vision-language-action models with data generation via residual RL," *arXiv preprint arXiv:2511.00091*, 2025. [Online]. Available: https://arxiv.org/abs/2511.00091

[3] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025. [Online]. Available: https://arxiv.org/abs/2502.19645

[4] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, D. Wang, D. Luo, Y. Fan, Y. Sun, J. Zeng, J. Pang, S. Zhang, Y. Wang, Y. Mu, B. Zhou, and N. Ding, "SimpleVLA-RL: Scaling VLA training via reinforcement learning," *arXiv preprint arXiv:2509.09674*, 2025. [Online]. Available: https://arxiv.org/abs/2509.09674

[5] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024. [Online]. Available: https://arxiv.org/abs/2402.03300

[6] Q. Yu, Z. Zhang, R. Zhu *et al.*, "DAPO: An open-source LLM reinforcement learning system at scale," *arXiv preprint arXiv:2503.14476*, 2025. [Online]. Available: https://arxiv.org/abs/2503.14476

[7] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen, "Improving vision-language-action model with online reinforcement learning," *arXiv preprint arXiv:2501.16664*, 2025. [Online]. Available: https://arxiv.org/abs/2501.16664

[8] J. Oh, Y. Guo, S. Singh, and H. Lee, "Self-imitation learning," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 3878–3887. [Online]. Available: https://proceedings.mlr.press/v80/oh18b.html

[9] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, "SmolVLA: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025. [Online]. Available: https://arxiv.org/abs/2506.01844

[10] D. Davis and B. Recht, "What is the objective of reasoning with reinforcement learning?" *arXiv preprint arXiv:2510.13651*, 2025. [Online]. Available: https://arxiv.org/abs/2510.13651